

SUMMER COURSE

Linux Development Foundations

Week 0 · Linux 開發環境入門

獵奇緯



目錄

01 課前導覽

03 桌面環境與顯示系統

05 檔案系統

07 套件管理與軟體安裝

09 程序、服務與 systemd

02 Linux 生態與發行版

04 Terminal 與 Shell

06 使用者、群組與權限

08 PATH 與環境變數

SECTION 01

課前導覽



01

碩士剛進實驗室沒有基礎，是很正常的。

Week 0 的定位

這是正式課程前的 **開發環境入門**，目標是：

- 建立一套 **看得懂系統的心智模型**
- 以 **日常工作流** 為範圍
- 奠定日常研究除錯的能力基礎

① Info

先修假設：修過作業系統，具備基礎觀念。

研究所錄取不是用雞腿換來的。



一句話

Linux 不神秘。

通靈大戰不適合我們這種靈能低下的凡夫俗子，我們 debug 還是乖乖找線索吧。

完成這堂課，你應該能夠

分得清楚

- kernel、發行版、桌面環境、terminal 與 shell 的差異
- 軟體、套件的不同種安裝方法
- PATH、環境變數、`source` 與 shell startup file

動得了手

- 查系統版本、架構、使用者、shell、桌面 session
- 用 terminal 完成檔案操作、搜尋與 log 檢查
- 讀懂使用者、群組、檔案權限與 sudo
- 觀察 process、port、service 與 systemd log



SECTION 02

Linux 生態與發行版



02

■ 當我們說「Linux」，到底在指什麼？

DISTRIBUTION

把 kernel 和 user space (userland) **選好、整合、打包、測試**，並提供更新與支援的系統產品 —— Ubuntu、Fedora、Arch...

USERLAND

shell、compiler、library、coreutils、service、desktop、log system 等使用者空間工具

KERNEL

管理硬體、process、memory、filesystem、network、driver 與 system call



課本沒教的是 —— **發行版到底是什麼**。

一段歷史：kernel 和 userland 從出身就是兩家人

1970s	UNIX 定下強勢慣例：多使用者、檔案抽象、小工具 pipe 組合、文字檔設定
1980s	UNIX 商業化、原始碼不再流通 → GNU project 立志重寫自由的 UNIX-like 系統
~1990	GNU 做齊 userland (gcc、bash、coreutils、glibc) —— 獨缺 kernel
1991-92	Linus 發表 Linux kernel、改採 GPL → 兩個世界 授權相容 ，可以合法組裝
1992-93	Slackware、Debian 誕生 —— 「把它組起來」的角色： 發行版



發行版賣的是什麼？

一組 **整合決策**，加上替這些決策 **負責的承諾**。

- 預設採用哪些 system component
- 套件格式與套件管理器
- 套件版本偏新還是偏穩
- rolling release 還是 fixed release
- kernel / driver 更新策略
- 安全機制、服務管理、log 預設
- 文件、社群、商業支援、長期維護



同一件事，多種方言

```
sudo apt install build-essential      # Ubuntu
sudo pacman -S base-devel             # Arch
sudo dnf groupinstall "Development Tools" # Fedora
```

選發行版 = 選「要跟誰的品味和維護策略」。

發行版家族

家族	成員與性格	套件	
Debian 系	Debian 保守可靠；Ubuntu = Debian 快照 + 硬體支援 + 半年一版	<code>.deb</code>	穩定與自由軟體傳統
Red Hat 系	RHEL 賣企業合約；Fedora 是前沿實驗場；Rocky / Alma 補免費相容位	<code>.rpm</code>	企業機房常客
Arch 系	簡潔、透明、rolling release，滾壞摸摸鼻子自己修	<code>.pkg.tar.zst</code>	"I use Arch btw"
特化選手	Alpine（musl、container 寵兒）、NixOS（宣告式）、Kali（資安工具箱）	—	

✓ Tip

Arch Wiki 是全 Linux 品質最高的文件 —— **就算不用 Arch 也很推薦看 Arch Wiki**。

這台機器是什麼？

```
uname -a          # kernel 版本、CPU 架構
cat /etc/os-release # 發行版：ID=ubuntu ...
lsb_release -a
hostnamectl
```

兩個指令答的是 **不同層**：同一版 Ubuntu 可以搭不同 kernel —— 正好驗證三層模型。

套件管理器是一本帳

`apt install git` 不只把 binary 放進 `/usr/bin/git`，還 **記下這個檔案屬於誰、由誰更新**。

每種安裝方式都有 **自己的管理邊界**。

Info

把三層模型放在心裡：後面的桌面、shell、檔案系統、權限、PATH、systemd，全都掛得回這個骨架。

SECTION 03

桌面環境與顯示系統



03

世界上絕大多數的 Linux 沒有螢幕

沿著開機路徑走一遍，看桌面出現得有多**晚**：



✓ 到 PID 1 為止

系統已經**完整**：能跑 process、管檔案、開網路、起 service。



伺服器 / 機器人 / NAS

一輩子停在這一層，在純文字環境裡活得好好的。



桌面

很上層、**可拆**的一塊。

TTY：桌面掛掉時的逃生門

名字來自 teletype（真的電傳打字機），現在指 virtual console。

現在就試：`Ctrl` + `Alt` + `F3`

等下記得切回來 (`F1` / `F2`)

✓ Tip

桌面掛掉時，TTY 往往還活著。



救援常見劇本

- 1 畫面凍結、GPU driver 發瘋 ...
- 2 切到 TTY，登入
- 3 殺掉出問題的 process、看 log，系統救回來

■ 桌面環境（DE）是「整合層」

一個 DE 打包了：window manager、panel、launcher、settings daemon、file manager、通知、鎖屏、輸入法、電源管理。

GNOME

一致與簡化路線。Ubuntu 預設使用客製化過的 GNOME。

KDE Plasma

什麼都能調，自由度很高，適合想控制每個細節的人。

XFCE / LXQt

輕量穩定，老機器和遠端桌面的常客。

❶ Info

差異都在**使用體驗**，底下仍同一顆 kernel。同一台機器可裝多個 DE，登入時選想要的 session 就好。

另一派玩法：只裝 window manager，自己拼

角色	X11 經典組	Wayland 人氣組
WM / compositor	i3 (tiling)	Hyprland (華麗)、sway (i3 後繼)
狀態列	polybar	waybar
launcher	rofi	wofi / fuzzel
通知 / 桌布	dunst / feh	mako / swaybg
截圖 / 鎖屏	—	grim + slurp / swaylock

⚠ 注意

但是 Wi-Fi、音量、亮度、輸入法、螢幕分享.....全要自己接，壞了無人可怪。
這類玩法解釋了 Linux 桌面的**可組裝性**，但我們日常研究不用它也完全沒差。

■ 桌面底下還有一層：顯示系統

display server / compositor：讓程式把畫面畫上螢幕、分發鍵盤滑鼠 input。

X11 · 1984

- network transparency： `ssh -X` 把遠端視窗開在本地
- `DISPLAY` 環境變數是它的遺產
- 四十年複雜度 + 安全問題（視窗可互相監聽鍵盤）

Wayland · 接班

- compositor 放核心位置
- 改善安全、HiDPI、觸控、frame timing
- GNOME / KDE 已預設；截圖、螢幕分享的陣痛是真的

登入畫面是 **display manager**：選使用者、session。

各位手邊有 **Linux** 開發環境了嗎？

觀察：我現在活在哪個世界？

```
echo $XDG_CURRENT_DESKTOP # GNOME / KDE /  
sway  
echo $XDG_SESSION_TYPE   # x11 / wayland  
echo $DISPLAY             # X11 的座標  
echo $WAYLAND_DISPLAY     # Wayland socket  
loginctl                  # 所有 login session
```

🎯 對開發者的兩個實戰價值

- 1 遠端伺服器幾乎都是純文字環境 —— SSH
開 GUI 撞上 `DISPLAY` / X forwarding 是必然
- 2 圖形程式打不開，檢查順序：**session → display protocol → permission → driver**

SECTION 04

Terminal 與 Shell



04

黑色視窗裡其實有兩層



(螢幕方向反著走同一條線)



terminal emulator

桌面上的普通應用程式（GNOME Terminal、Alacritty、Kitty...）。「模擬」當年一人一台的實體終端機：顯示文字、收輸入、處理控制序列。



shell

在 terminal「裡面」跑的程式：讀你打的字、解析指令、展開 wildcard、接 pipe 與 redirection、啟動別的程式。

✓ Tip

`vim` / `less` / `top` crash 後 terminal 壞掉（打字不顯示、畫面異常）？

盲打 `reset` + Enter

一行指令，shell 做了多少事

```
ls -lah | grep ".py" > files.txt
```

- 1 拆 token：ls、-lah、pipe、grep、字串、redirection
- 2 生出兩個 process
- 3 第一個的 stdout 接到第二個的 stdin
- 4 第二個的 stdout 導進 files.txt



關鍵認知

ls 和 grep **不知道 pipe 存在** —— 管線是 shell 在啟動它們之前偷偷接好的。

UNIX 哲學的具體實現：程式只管讀 stdin 寫 stdout，**組合是 shell 的事**。

Shell 地雷區

variable、condition、loop、function、exit status、quoting rule 一應俱全。

慘案迷因

`rm -rf "$DIR/"` —— 變數沒設到，展開成 `rm -rf "/"`

寫 script 的三個**偏執習慣**：

- 1 變數永遠加雙引號
- 2 動手前先 `echo` 看展開結果
- 3 寫完丟 `shellcheck`

| Bash: interactive & non-interactive

Bash : GNU 的 shell、多數發行版預設，
POSIX 相容 + history / completion /
array / job control 。

```
export PATH="$HOME/.local/bin:$PATH"  
source ~/.bashrc  
for f in *.txt; do  
    echo "$f"  
done
```

🧱 最容易撞牆的區分

- **interactive** : 打開 terminal 手動輸入 → 讀 `~/.bashrc`
- **non-interactive** : `bash script.sh` → 規則不同
- login shell 又是另一組 (`~/.profile` 、 `~/.bash_profile`)

⚠ 注意

「terminal 有這指令、script 裡找不到」
九成是**設定寫在不會被讀的檔案**。

Shell 圖鑑

Shell	定位	常見事項
<code>zsh</code>	開發者桌面人氣王（macOS 預設）、completion / prompt 生態豐富	interactive 用 zsh 爽， script 則寫 Bash
<code>fish</code>	開箱即用 autosuggestion、syntax highlighting	語法刻意非 POSIX，網路上的 Bash 片段貼進去常報錯
<code>dash</code>	小而快的純 POSIX；Ubuntu 的 <code>/bin/sh</code> 就是它	<code>#!/bin/sh</code> + Bash 語法可能會出錯

× 危險

要用 Bash feature 就明寫 `#!/usr/bin/env bash`。寫 `#!/bin/sh` 只准用 POSIX 語法。

SECTION 05

檔案系統



05

| Everything is a file

UNIX 的標誌性設計哲學：

學會**讀寫檔案**，就能觀察和操作幾乎整個系統。

- 鍵盤滑鼠硬碟 → `/dev` 裡是檔案
- process 狀態 → `/proc` 裡是檔案
- kernel / 硬體狀態 → `/sys` 裡是檔案



預設工具

```
ls /proc/<pid>/fd    # 它開了哪些檔案  
cat /proc/cpuinfo    # CPU 資訊
```

檔案系統本身就是系統的儀表板。

一棵樹，沒有 `c:\` —— 全靠 `mount` 拼起來

根是 `/`，**只有一棵樹**。那我第二顆硬碟去哪了？被 `mount` 進樹裡了。

不同磁碟、分割區、網路 / 虛擬檔案系統，全接到某個目錄上，組成單一 namespace。

```
mount ; findmnt    # 目前的 mount tree
df -h             # 各 filesystem 容量
lsblk             # block device 與分割區
```



鬧鬼現場

「檔案怎麼不見了／怎麼多出來了」——

`mount` 可以把原本目錄裡的東西 **蓋住**，`umount` 之後又冒出來。

先 `findmnt` 確認自己站在哪個 filesystem 上。

目錄慣例

路徑	住著什麼
<code>/bin</code> <code>/usr/bin</code>	一般使用者可執行程式
<code>/sbin</code> <code>/usr/sbin</code>	系統管理程式
<code>/etc</code>	系統設定檔
<code>/home</code> <code>/root</code>	家目錄（一般 / root）
<code>/var</code>	會變動的資料：log、cache、db state
<code>/tmp</code>	暫存，重開機通常就沒
<code>/opt</code>	第三方巨獸（CUDA、ROS）

路徑	住著什麼
<code>/usr</code>	多數 userland 程式、library、文件
<code>/dev</code>	device file
<code>/proc</code> <code>/sys</code>	process / kernel 的虛擬檔案系統
<code>/run</code>	開機後的 runtime state(socket、pid)

反射動作

- 設定 → `/etc`
- log → `/var/log`
- 個人設定 → `$HOME` dotfiles
- 硬碟滿了先懷疑 → `/var`

I 路徑與基本操作

```
pwd                # 我在哪
ls -lah            # 權限、owner、隱藏檔
cd /path/to/dir
mkdir -p build/logs # 一口氣建多層
cp source target
mv old new
rm file
find . -name "*.py" # 找「檔案」
```

找「檔案內容」用 `grep` / `rg` —— 分工要清楚。

- **absolute**：從 `/` 開始
- **relative**：相對目前工作目錄；`.` 這裡、`..` 上層、`~` home

⚠ 注意

relative path 相對的是 **process 的工作目錄**，不是 **script 所在位置**。

× 危險

打 `rm -rf` 前停一拍。不放心就加 `-v` 看它刪什麼。

每個檔案都帶著 metadata

-rw-r--r--

類型：- 檔案 d 目錄 l symlink

OWNER rwx

GROUP rwx

OTHERS rwx

owner、group、size、mtime、inode —— stat 全看得到。

⚠ 注意

對 目錄 來說，execute 的真正意義是 **traverse**（能不能穿過）。

路徑上某層目錄沒給 x，檔案本身有 r 也讀不到。

| Symlink、dotfiles，與檔案系統的「型號」

symlink

```
ln -s target link_name  
readlink link_name
```

版本切換頻繁，但只需要換 symlink。方便，不過要注意每可能指歪。

dotfiles

檔名以 `.` 開頭就隱藏，純粹只是命名慣例（源自 `ls` 一個太寬鬆的判斷）。

把 dotfiles 放進 git repo，換機器一拉就是熟悉環境。

型號

本機 ext4 / xfs / btrfs、隨身碟 exFAT / FAT32、網路 NFS。

路徑長得一樣，底層行為可能不同 —— FAT32 沒有權限、沒有 symlink，不是壞了。

SECTION 06

使用者、群組與權限



06

I root 是什麼？

1970 年代：一台昂貴主機、幾十人分時共用 → 系統必須知道 **這是誰、他能碰什麼**。

時至今日就算電腦只有一個人用，照樣運轉同樣的模型。

每個使用者一個 **uid**、每個群組一個 **gid**；名字給人看，**kernel 認數字**。

uid 0 = root

root 不是「權限比較多」的使用者，
是「**權限檢查對他不生效**」的使用者。

拿到 root = 拿到整台機器。

正確態度：**敬而遠之**，需要時短暫借用。

群組，權限的批發機制

```
whoami
id      # uid、gid、所有附屬群組
groups
```

與其一個個使用者授權，不如把權限給群組、再把人加進群組：

```
sudo usermod -aG dialout $USER
```

- `/dev/ttyUSB0` 代表透過 USB 介面連接至電腦的「USB 轉序列埠」裝置，常屬於 `dialout`
- Docker 想要設置為免 sudo，要把自己加進 `docker` 群組

× 危險

`-aG` 忘了 `a` (append) → 附屬群組全部洗掉。

⚠ 注意

群組變更要重新登入才生效。

① Info

加入 `docker` 群組 ≈ 免密碼 root。

解碼 **rwX**

檢查順序：

- 是 owner → 用第一組
- 在 group → 第二組
- 都不是 → 第三組

只用命中的那組。

r=4、w=2、x=1

數字	展開	慣例用途
755	rwXr-xr-x	執行檔、目錄
644	rw-r--r--	一般檔案
777	rwXrwxrwx	 布瑠部，由良由良...

× 關於 **chmod 777**

意思是「這台機器上**任何程式**都能改寫這個檔案」，基本等同於把門拆了。

正確動作永遠是搞清楚**誰需要什麼權限**，

`chown` 調 owner、`chmod` 給最小必要權限。

```
chmod 644 file
chown user:group file
umask # 新檔案的預設權限遮罩
```

三個特殊權限位

setuid · s

`passwd` 憑什麼寫

`/etc/shadow` ? 執行時**暫時以檔案 owner (root) 身分跑**。

最精巧也最危險的機關。很多提權漏洞出在這。

sticky · t

`ls -ld /tmp` 尾巴的 `t` : 大家都可寫，但**只有 owner 能刪自己的檔案**。

不然共用 `/tmp` 大家互刪，天下大亂。

setgid (目錄)

新檔案 **繼承目錄的 group**。

共用專案目錄常用。

✓ Tip

看到權限欄出現 `s` 和 `t`，要知道是什麼就好。

| sudo：有紀錄的借用，不是變身



su VS sudo

su：變身成 root，然後一直是 root。

sudo：借一下力量，用完就還。

sudo 不共享 root 密碼、每條指令有 log、`/etc/sudoers` 可細分授權。



品味判斷

看到 `Permission denied` 就無腦補 sudo —— 最該戒掉的反射。

- 裝系統套件、動 `/etc`、開 1024 以下 port → 合理
- 在自己 home 寫檔要 sudo → 有東西壞了，修那個

× 危險

`sudo pip install` 是禁忌.....

SECTION 07

套件管理與軟體安裝



07

I 1990 年代：dependency hell

當年的標準發行方式：source tarball —— 下載、解壓、

```
./configure && make && make install
```

。

💣 致命傷一：沒有紀錄

檔案散進系統各處，沒人知道哪個檔案屬於哪個軟體 —— 移除靠猜、升級靠緣分。

💩 致命傷二：依賴人肉追

A 要 libB 1.2 以上、libB 又依賴 libC..... 追錯一層就編不過。

發行版套件系統的解法：帶 metadata 的 **package** + 記錄檔案歸屬的 **資料庫** + 集中發行、簽章驗證的 **repository**。

■ 那為什麼今天還是亂？



套件系統要的：系統整合

全機一致、穩定、可維護 —— 版本由發行版決定、全機只有一份。



開發要的：剛好相反

要新版本、要一個專案一個版本、要不用 root 就能裝。

於是各語言生態長出自己的套件管理器 —— **每套各自合理，但誰也不認得別套的帳。**

今天的亂，來自**好幾套各自沒錯的系統思路疊在同一個 *filesystem* 上。**

I deb & dpkg

`.deb` 只是一個封存檔，拆開兩塊：

- **control**：metadata —— 名字、版本、依賴、preinst / postinst 維護腳本
- **data**：真正要鋪進 `/` 的檔案樹

```
dpkg-deb -I some.deb # 看 metadata 與依賴
dpkg-deb -c some.deb # 看檔案會鋪到哪
```

⚙️ dpkg 做三件事

鋪檔案 → 登記進 `/var/lib/dpkg` 資料庫 → 跑維護腳本

和兩個重要的「不會」

不會上網下載、不會解依賴。777

`dpkg -i` 卡依賴，正是分工到此為止的表現。

apt

```
sudo apt update           # 只更新套件索引，不升級任何軟體
sudo apt upgrade          # 真正升級
sudo apt install btop      # 解依賴、下載、驗簽，最後交給 dpkg
sudo apt install ./some.deb # 對本地 deb 也能順便解依賴
dpkg -L btop               # 查帳：這個套件擁有哪些檔案
dpkg -S /usr/bin/btop      # 反查：這個檔案屬於哪個套件
```

✓ Tip

`dpkg -i` 的殘局 → `sudo apt -f install`
收拾。

① Info

雙向查帳，套件系統的核心價值。限制也同源：版本偏舊、全機一份。

I 語言生態（1）：pip 的事故現場

pip 和 apt **都認為自己管 Python 套件**，歷史還寫同一片目錄。當我們

`sudo pip install` 之後就可能發生兩本帳對不上的情況，恭喜你的環境就出事了。

❶ Info

新版 Ubuntu 的

`externally-managed-environment` 錯誤，
正是在擋事故。

正解只有一條：**Python 永遠裝在 venv 或使用者層級**。

```
uv venv          # 專案內建立隔離環境
uv add numpy     # 裝進環境，跟系統無關
uv tool install ruff # 全域工具，各住一個私有環境
```

`uv tool`：工具裝進獨立環境、指令 symlink 到
`~/.local/bin` 帳本齊全：`list` / `upgrade` /
`uninstall`。

這門課本人推薦使用 `uv`，不過很多研究會偏好用
`conda`。

I 語言生態 (2) : cargo 的世界



`cargo install`

抓原始碼 → 本機編譯 → binary 進

`~/.cargo/bin` 。

Rust binary 幾乎不依賴外部函式庫，不需要隔離環境。帳用 `cargo install --list` 查，升級要自己重跑。



很類似的其他語言生態

`npm install -g` 、 `go install` 是變體。

共同精神：**裝在 home、不碰系統地盤、不需要 sudo**。

× 危險

看到教學對 **語言套件** 下 `sudo` 不要傻傻照做。

真正的高風險區：`sudo make install`

C/C++ 沒有預設套件管理器，大量專案至今仍是原始碼 + `sudo make install`。風險高一級的本質：**裝的常常是函式庫，而函式庫全系統共享**。

1 · 沒有帳本

bin / lib / include 散進

`/usr/local`，沒人記錄。唯一收據 `install_manifest.txt` 躺在 build 目錄——刪了就沒了。

2 · Shadow 全機函式庫

`/usr/local/lib` 優先於

`/usr/lib` —— 自編

libopencv / libprotobuf 蓋掉發行版那份，**全機**程式一起吃錯版本。

3 · prefix 錯，跟 apt 打架

裝進 `/usr` 就是覆寫發行版檔案，apt 升級又會再蓋回來。許多天後，你還有把握系統最後狀態是怎樣嗎？

I 正確習慣 + 散裝流派

- 1 有套件就用套件 —— apt、官方 deb、PPA。原始碼編譯是最後手段
- 2 真要編，**不要用預設 prefix**：`-DCMAKE_INSTALL_PREFIX=$HOME/.local` 或版本化進
`/opt/opencv-4.10`。下游用 `CMAKE_PREFIX_PATH` 指過去
- 3 留收據：build 目錄與 `install_manifest.txt` 不刪，或用 `checkinstall` 包成 deb
- 4 最乾脆：不 install —— build tree 內連結，或整個進容器

散裝流派，認得就好

binary / AppImage 丟 `~/.local/bin`（更新自己來）· `curl ... | bash`（rustup、uv 都這樣發行）·
snap / flatpak（沙盒路線）—— **每一種都是一本自己的帳**。

容器化

```
docker pull ubuntu:24.04
docker run -it --rm ubuntu:24.04 bash
```

進去是另一個世界：另一套 `/usr/bin`、另一本 apt 帳，跟 host 互不相干。

對 dependency hell 的解法：**不解決衝突，直接讓大家不見面**。

Info

我們下週會再介紹容器.....敬請期待



機器人開發的現實

ROS 對 Ubuntu 版本綁定很緊，很多實驗室的標準做法：

host 保持乾淨，開發環境全部進 container。

I 重點整理

安裝方式	地盤	查帳	注意
<code>apt</code>	<code>/usr</code> 全系統	<code>dpkg -L</code> / <code>dpkg -S</code>	版本偏舊、全機一份
<code>uv</code> / <code>pip</code> (venv)	venv / <code>~/.local</code>	<code>uv tool list</code> 、 <code>pip list</code>	永遠不要 <code>sudo pip</code>
<code>cargo</code> / <code>npm -g</code> / <code>go</code>	<code>~/.cargo/bin</code> 等	<code>cargo install --list</code>	升級自己重跑
<code>make install</code>	<code>/usr/local</code> (預設)	<code>install_manifest.txt</code> (僅存收據)	自訂 prefix、留收據
binary / AppImage	<code>~/.local/bin</code>	自己記	更新自己來
Docker	image 裡的世界	<code>docker images</code>	跟 host 不見面

SECTION 08

PATH 與環境變數



08

經典症狀

我們以後課程進到 ROS 2 之後，**每開一個新 terminal** 往往就要

```
source /opt/ros/<distro>/setup.bash
source install/setup.bash # 自己的 workspace
```

為什麼呢？如果沒有做的話.....

症狀一

```
ros2: command not found
```

症狀二

```
Package 'xxx' not found
```

(明明剛編譯成功)

症狀三

跑起來了，但用的是**舊版本**

根源是 **Linux 環境變數**

■ 症狀一的謎底：PATH，先到先贏

shell 不搜尋整個硬碟，它看 `PATH`：

```
echo $PATH  
# /home/rvl/.local/bin:/usr/local/bin:/usr/  
bin:...
```

從左到右一個一個找，**找到第一個就停**。



謎底

ROS 2 住在 `/opt/ros/<distro>`，它的 bin **不在預設 PATH 裡**。

程式好好躺在硬碟上，只是 shell 的查找清單裡 **沒有它的地址**。

`setup.bash` 做的第一件事：把這個目錄加進 PATH。

查案三支：`which ros2`（會選中哪個）、`type -a python3`（同名全列出）、`command -v`。
Bash 有指令位置快取。

「裝了新的還跑到舊的」用 `hash -r` 清。

| setup.bash 改的遠不只 PATH

變數	誰用它、找什麼
PATH	shell 找可執行檔
AMENT_PREFIX_PATH	ros2 指令找 package 索引 —— 症狀二的謎底
PYTHONPATH	Python 找 ROS 模組
LD_LIBRARY_PATH	程式啟動時找動態函式庫
CMAKE_PREFIX_PATH	編譯時找依賴

它們全是環境變數，一組跟著 process 走的 key-value。

傳遞規則：fork + exec，複製而非共享

各位回憶作業系統所學：新 process 會

fork + exec。

- 環境變數存在 process 自己的記憶體 (environ) → fork 時 **跟著複製**
- exec 換程式，預設 **原封不動帶過去**
- 子 process 再怎麼改自己那份，**父 process 一個位元組都不動**

shell 每執行一個指令就是一次 fork + exec。

整棵 process tree = 環境變數的繼承樹

```
systemd (PID 1)
├── gdm - session - terminal - bash ← 你 source 的在這
│   └── ros2 run ... (繼承自 bash)
└── sshd / cron - bash -c script ← 另一條鏈，感覺不到
```

I 情境一：為什麼要 `source`，不能直接執行？



```
bash setup.bash
```

fork + exec 一個 **子 shell** → 子 shell 在自己的記憶體設好變數 → exit → 記憶體隨 process 消失。

父 shell **什麼都沒得到**。



```
source setup.bash
```

不 fork —— 當前 shell 親自逐行執行檔案內容，變數寫在 **自己身上**。

① Info

這也回答了「為什麼每開新 terminal 都要重 source」：每個 terminal 是全新 shell process，環境是 **出生那一刻複製來的**，跟上一個 terminal 毫無關係。

I 情境二：換條鏈就掛——systemd、cron、SSH

terminal 跑得好好的 node，寫成開機自動啟動或 SSH 進去跑就掛，就是 Shell 那節提的 **PATH 靈異事件**。

病理・上半：繼承鏈

systemd / cron 生的 process 祖先是 PID 1，父鏈 **不經過你的 interactive shell**。

繼承只沿父子鏈走。

你 source 過什麼，**別條鏈上一個位元組都感覺不到**。

所以機器人的 bringup 腳本

開頭總要自己 `source` 一次 setup 檔，或在 service 設定裡明確給環境。

病理・下半（哪些檔案什麼時候被讀）。

I 情境三： `export` ，與平行世界的 node

shell 有兩種變數：

- 純 shell 變數：只活在 shell 自己的變數表，**不會** 進交接給子 process 的 environ
- `export`：把它標記進交接清單

```
ROS_DOMAIN_ID=42      # node 看不到  
export ROS_DOMAIN_ID=42 # 傳下去 ✓
```



專屬靈異現象

兩個 terminal 一個有 `export`、一個沒有 → node 活在 **兩個平行世界**，`ros2 topic list` 互相看不到。

症狀像網路壞了，實際只是兩個 shell 的 environ 不同。

聯調前各自跑 `env | grep ROS`，省掉互相懷疑。

症狀三「跑到舊版本」同理：workspace 的 setup 把自己疊在 `/opt/ros` **前面**（ROS 叫 overlay，原理就是先到先贏）。Python venv 的 activate 也是同一招。

病理・下半：startup file 對照表

shell 情境	例子	讀什麼
login shell	SSH 登入、TTY 登入	<code>~/.profile</code> / <code>~/.bash_profile</code>
interactive non-login	桌面上開 terminal（最常見）	<code>~/.bashrc</code>
non-interactive	跑 script、systemd 起的程式	基本上 什麼都不讀

✓ Tip

實務守則：環境設定寫進 `~/.bashrc`，並確認 `~/.profile` 有引用它（Ubuntu 預設有；鏈斷了就出現「SSH 進來 ros2 就消失」）。

× 危險

```
export PATH="$HOME/.local/bin:$PATH"
```

—— 忘記接 `$PATH` 會把整條路清空，連 `ls` 都 command not found。

SECTION 09

程序、服務與 systemd



09

I 收官：觀察活著的系統

前面全是靜態結構（檔案、權限、路徑），但我們除錯面對的是**活的系統**。

```
ps aux          # 全量快照，配 grep
ps auxf         # 加家族樹縮排
top             # 活的儀表板
btop            # 現代版，好看好操作
pstree          # 誰生了誰
```

✚ 前面每一節在這裡會合

每個 process 有 PID、父 process、**owner**（權限那節的身分）、**自己的環境變數**（上一節的繼承）。

`btop` 要自己裝 (`sudo apt install btop`) 。

kill 這名字取得很兇，其實只是「送訊號」

```
kill PID           # SIGTERM，好好講
kill -9 PID        # SIGKILL，不講了
pkill -f pattern
```

SIGTERM	「請收拾好自行了斷」——可攔截、寫檔、清理再走
---------	-------------------------

SIGKILL	kernel 直接抹名冊—— 連遺言都來不及留
---------	--------------------------------

SIGINT	就是天天在按的 <code>Ctrl</code> + <code>C</code>
--------	--

規矩：先 `kill`，等幾秒，真不走再 `-9`。

❶ 破迷思：zombie

zombie 用 `kill -9` 殺不掉——它 **已經死了**，只是一筆等父親收屍（讀 exit status）的戶口紀錄。父親不收，得找父親算帳。

SSH 斷線，程式死光光

為什麼？程式是 SSH session 的 shell 的子孫 —— session 斷掉，整個家族樹收到一記 **SIGHUP**（hangup，來自電話掛斷的年代），預設行為就是跟著死。



土法

`nohup` —— 讓程式無視掛斷訊號。



日常必備

`tmux` / `screen` —— 斷線後還活著、重連接得回去的 session。

遠端機器進門先開 `tmux`，值得練成肌肉記憶。



正規解

交給系統的服務管理員看護 —— 本節後半的主角，先按下不表。

Port : process 在網路上的門牌

IP 找到 **房子**（哪台機器），port 找到 **房間**（哪個 process）—— 16-bit 號碼（0-65535），程式先向 kernel 登記「我要聽 8080」（listen），之後打到 8080 的封包全歸它。

推論 1

同一 port 同時只能一個 process 聽 —— 後到的拿到

`Address already in use`。

推論 2

listen 還要選介面：

`127.0.0.1` 只收本機；

`0.0.0.0` 所有介面都收。

推論 3

1024 以下 = privileged port，要 root（22、80/443）—— 所以測試伺服器預設 8000、8080。

Port 查案

```
ss -tlnp          # 誰在聽哪些 TCP port
ss -tulnp         # 連 UDP 一起列
sudo lsof -i :8080 # 直接問：誰佔了 8080
curl localhost:8080 # 本機戳一下，確認活著
```

(老教學寫 `netstat`，同一件事的老工具。)

 輸出現在看得懂了

`127.0.0.1:8080` VS `0.0.0.0:8080` = 門牌掛哪張網卡。

查到佔用 PID → 殺它，或自己換 port。

① Info

ROS 2 的 DDS 走 **UDP** ——

`ros2 topic` 流量在 TCP 清單看不到，要 `-u` 才現形。

■ PID 1：kernel 只認一個初始 process

機器上一大堆程式不是你開的：SSH server、network manager、Docker daemon.....
開機自己起、掛了有人重拉、log 有人收 —— 這個角色叫 **init system**。

👑 PID 1 有多特殊

- kernel 開機只啟動 **唯一** 一個初始 process，其後全是它的子孫
- 它死了 → kernel 直接 panic
- 孤兒 process 最後都過繼給它收屍

👤 前任：sysvinit (1980s 風格)

按順序跑 `/etc/init.d/` 的 shell script：開機慢、腳本品質參差、daemon double fork 逃出管轄 —— **stop 常停不乾淨**。

| systemd 的三個核心翻新（2010，參考 launchd）

1 · 宣告式 unit 檔

「執行哪個程式、依賴誰、掛了怎麼辦」寫成純文字設定——又見「文字檔當設定介面」的老傳統。

2 · 依賴圖平行啟動

從排隊執行變成依賴圖上平行起——開機速度換了一個世代。

3 · cgroup 記帳

服務生出的**每一個** process 圈在同組——double fork 也逃不掉，stop 是真的全停；`systemctl status` 的 process tree 靠這個。

■ 軼聞：近十五年吵得最兇的技術戰爭

引爆點：systemd 把分散的系統管理問題 **收進同一套模型** —— 服務 → unit、log → journal、login → logind、排程 → timer、socket activation.....

支持者

一致的依賴、狀態和查詢介面。

反對者

違反 UNIX 「一個程式做好一件事」的版圖擴張。

激烈到什麼程度：

- 2014 Debian 技術委員會投票吵到 **靠主席決定票** 定案
- 反對派憤而出走，fork 出拔掉 systemd 的 **Devuan**

本質是「鬆散組合 vs 集中協調」的路線之爭。結局：Fedora 2011 → Debian / Ubuntu 2015 → 現在整個現場圍著它轉。

Unit 與 systemctl 日常

unit 種類：`.service` 服務、`.timer` 排程、`.socket` 連線才啟動、`.target` 集合點（`multi-user.target` 純文字層 / `graphical.target` 疊桌面）。

兩個家、後者優先：`/usr/lib/systemd/system/`（套件裝的）、`/etc/systemd/system/`（管理者覆寫）——跟 `/etc` 精神一脈相承。

```
systemctl status ssh      # 第一反應指令
sudo systemctl start ssh
sudo systemctl restart ssh
sudo systemctl enable --now ssh
systemctl list-units --type=service
systemctl cat ssh         # 印出 unit 設定
```

⚠ 注意

經典地雷：`start` 和 `enable` 是兩件事 —— `start` 是現在跑，`enable` 是開機自動跑。
要又跑又自動：`enable --now`。

journalctl：全系統 log 的統一查詢

journald 把服務的 stdout / stderr、kernel 訊息全收進同一個資料庫：

```
journalctl -u ssh           # 指定 service
journalctl -u ssh -f        # follow：掛一個視窗盯著滾
journalctl -b               # 只看本次開機
journalctl -p err -b        # 只看錯誤等級以上
journalctl --since "10 min ago"
```

✓ Tip

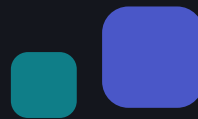
這幾個組合起來，「服務為什麼掛了」的答案 **九成能自己找到**。

① Info

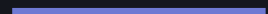
傳統 `/var/log`（`syslog`、`dmesg`、各應用 log）也還在——兩邊都要知道去翻。

第 0 週課程地圖

節	心智模型	指令
生態	kernel / userland / 發行版三層	<code>uname -a</code> 、 <code>cat /etc/os-release</code>
桌面	桌面很晚、很可拆；session 是選的	<code>echo \$XDG_SESSION_TYPE</code>
shell	pipe 是 shell 接的水管	<code>type -a</code> 、 <code>reset</code>
檔案	一棵樹 + mount + symlink + metadata	<code>findmnt</code> 、 <code>stat</code> 、 <code>df -h</code>
權限	process 身分 × 檔案門禁，root 免檢	<code>id</code> 、 <code>ls -l</code>
套件	每種安裝方式一本帳	<code>dpkg -S</code> 、 <code>uv tool list</code>
PATH	環境變數 = 沿 fork 複製的繼承樹	<code>echo \$PATH</code> 、 <code>env grep ROS</code>
systemd	PID 1 看護一切、journal 收一切	<code>systemctl status</code> 、 <code>journalctl -u</code>



Q & A



獵奇緯 · Summer Course